Experiment 6

Title: Implementation of a Fixed-Size Hash Table with Division Method and Linear Probing

Objectives:

To design and implement a hash table data structure of a fixed size.

To utilize the division method for calculating hash indices.

To implement linear probing for resolving collisions during key insertion and search.

To enable users to perform essential hash table operations: insert, search, delete, and display.

Problem Statement:

Efficient storage and retrieval of data are crucial in many applications. This project addresses the need for a data structure that allows for fast average-case performance for insertion, deletion, and search operations, even with a fixed memory allocation. Collisions, a common issue in hashing, need to be effectively managed to maintain performance.

Outcomes:

A functional hash table implementation capable of storing integer keys, performing insertions, searches, deletions, and displaying its current state. The implementation will demonstrate the use of division method hashing and linear probing for collision resolution.

Software and Hardware Requirements:

Software:

C/C++ compiler (e.g., GCC), Text editor or Integrated Development Environment (IDE) (e.g., VS Code, Code::Blocks).

Hardware:

Standard personal computer with sufficient RAM and storage for development and execution.

Theory Concept in brief:

A hash table is a data structure that maps keys to values using a hash function. The division method for hashing calculates the hash index as key % table_size. Collision resolution is necessary when two different keys map to the same index. Linear probing is a collision resolution technique where, upon a collision, the algorithm sequentially searches for the next available empty slot in the table. Deletion typically involves marking a slot as "deleted" to maintain the integrity of subsequent searches.

Algorithm:

Code:-

```
// Hash Function (Division Method)
hash(key, table_size):
    return key % table_size

// Insert Key
insert(key, hash_table, table_size):
    index = hash(key, table_size)
    while hash_table[index] is not empty and hash_table[index] is not DELETED:
        index = (index + 1) % table_size
    hash_table[index] = key

// Search Key
```

```
search(key, hash_table, table_size):
    index = hash(key, table_size)
    start_index = index
    while hash_table[index] is not empty:
        if hash_table[index] == key:
            return index // Key found
        index = (index + 1) % table_size
        if index == start_index: // Full circle without finding
            break
    return -1 // Key not found

// Delete Key
delete(key, hash_table, table_size):
    index = search(key, hash_table, table_size)
    if index != -1:
        hash_table[index] = DELETED // Mark as deleted

// Display Table
display(hash_table, table_size):
    for i from 0 to table_size-1:
        print hash_table[i]
```

Flowchart:

Test Cases:

Insert:

Insert keys: 10, 20, 30 (assuming table_size = 5).

Expected: Keys placed at calculated indices or subsequent available slots.

Search:

Search for an existing key (e.g., 20).

Search for a non-existing key (e.g., 15).

Delete:

Delete an existing key (e.g., 10).

Attempt to delete a non-existing key (e.g., 25).

Verify subsequent searches for the deleted key.

Collision Handling:

Insert keys that cause collisions (e.g., 5, 10, 15 with table_size = 5).

Verify linear probing correctly places subsequent keys.

Display:

Display the table after various insert/delete operations.

Test Data Set (if applicable):

For a table_size = 7:

Insert: 5, 12, 19, 26, 3, 10, 17

Delete: 12, 10

Search: 19, 3, 25

Mathematical Model (if applicable):

The hash function is defined as

h(k)=k(mod m), where

■ is the key and

■ is the table size.

The probe sequence for linear probing is h(k),(h(k)+1)(modm),(h(k)+2)(modm),…h of k comma open paren h of k plus 1 close paren space open paren mod m close paren comma open paren h of k plus 2 close paren space open paren mod m close paren comma …

■(■),(■(■)+1)(mod■),(■(■)+2)(mod■),…

.

Conclusion/Analysis of Design and Implementation:

The design successfully implements a fixed-size hash table using the division method for hashing and linear probing for collision resolution. The system allows for basic hash table operations. While linear probing is simple to implement, it can suffer from primary clustering, potentially degrading performance for large numbers of collisions. Future improvements could explore alternative collision resolution strategies like quadratic probing or separate chaining to mitigate this issue. The fixed size also limits scalability, suggesting dynamic resizing as a potential enhancement for more robust applications.

Group D: Graphs and Trees